

内存受限 CPU/CPU-GPU 场景下 MoE 推理运行优化综述

重点问题是: 在 CPU-only、CPU 主导、或小显存 GPU 辅助的本地机器上, 如何让大型 MoE 模型可运行、可交互, 并尽量减少专家权重搬运、缓存失效和低比特精度损失。

结论摘要

当前内存受限 MoE 推理优化的主线不是单一技巧, 而是围绕同一个事实展开: MoE 每 token 只激活少量专家, 但部署时往往被迫保存全量专家权重。真正的瓶颈从纯计算转为分层内存系统中的搬运、缓存、预测和调度。

最值得采用的技术组合是:

1. **把常驻路径和专家路径分开处理**。attention、embedding、router、norm、shared expert 等 always-on 部分常驻并尽量保护精度; routed experts 用低比特、按 expert/layer 切片、可 mmap/stream 的格式保存。
2. **建立显式 expert residency/cache 层**。至少做 per-layer expert LRU/ARC; CPU+GPU 时加 GPU hot expert cache、pinned host arena、异步 copy stream; CPU-only 时利用 OS page cache、用户态 LRU 和顺序 readahead。
3. **用调度隐藏 I/O, 而不是只优化单个 kernel**。MoE-Lightning 的 CGOPipe、MegaTrain 的 double-buffer、MoE-Gen 的 module-based batching、CrossPool 的 layer-wise pipeline 都说明: 需要把加载、计算、KV、hidden-state transfer 和控制开销放到同一个调度模型里。
4. **用预测减少 cache miss, 但必须有置信度和回退策略**。DALI、Speculating Experts、colibri 的 router-lookahead、CommitMoE 都在利用下一层/未来 token 的 routing 可预测性。实用系统里应先记录命中率, 再按置信度区分 prefetch、commit execution、fallback。
5. **压缩专家是第一性需求**。FloE 的 intra-expert compression、InstinctRazor 的 INT3 expert/INT4 backbone/BF16 protected path、vllm-Moet 的 2-bit expert + FP4 recovery、PreMoE 的 PEU pruning 都指向同一件事: routed experts 是主要容量来源, 低比特和剪枝要优先作用在它们上面, 但 router/norm/shared path 不能乱压。
6. **speculative decoding 是第二阶段优化**。MTP/self-assisted speculative decoding 可以降低每输出 token 的完整 MoE 执行次数或搬运次数, 但冷 cache 时可能反而增加 expert load。应在基础 cache/pipeline 稳定后再接入。

如果我们也做这个方向, 最现实的切入点是一个 **CPU-first MoE runtime**: 先支持一个模型族, 做专家低比特离线转换、per-layer LRU/ARC、异步 readahead、batch-union prefill、CPU AVX/AMX kernel; 再加小 GPU hot cache、residual/router-lookahead prefetch、PEU/utility pruning 和 MTP/grammar speculation。

资源与相关性

这批材料可分为几组:

类别	代表工作	和本方向的关系
offloading 系统	MoE-Lightning, MoE-Infinity, mixtral-offloading, FloE, DALI, CommitMoE, elsa-lab CPU-GPU collaborative inference	直接相关，解决专家权重在 CPU/SSD/GPU 之间迁移的问题
CPU/本地推理工程	colibri, KTransformers, vllm-Moet	直接相关，包含低比特专家、CPU/GPU 异构、NVMe/pinned RAM/cache 等工程实现
预测与 speculation	Speculating Experts, SpecMoE 2604.10152, Juncail/SpecMoE, colibri MTP	相关，减少 token 级 MoE 执行或提前加载专家
pruning/quantization/router locality	PreMoE, ReMoE, InstinctRazor, FloE	相关，减少需要保存或搬运的专家容量，并提高 cache locality
serving/KV 分离	CrossPool, MoE-Infinity serving	对多模型/长上下文/冷模型服务相关
支撑性系统	MegaTrain, FastMoE, YALIS	MegaTrain 是训练系统但 double-buffer/host-memory-first 思路可迁移；FastMoE 更偏训练/分布式 MoE primitives；YALIS 是研究型 inference base
不纳入主线	TeraXj78/SpecMoE	这是 EEG foundation model 的 MoE，不是 LLM 推理优化

Zhihu 快照只有 650 字节，疑似访问限制页，因此没有把它作为实质来源。

核心瓶颈

MoE 的理想优势是每 token 只计算 top-k experts。但在本地部署时，常见模型的总专家权重远大于显存甚至超过普通内存预算。于是出现四个瓶颈:

1. **容量瓶颈**: 全量 expert weights 占用过大。Mixtral-8x7B FP16 约需 94GB VRAM, GLM/DeepSeek/Qwen 新 MoE 可到数百 GB。
2. **带宽瓶颈**: PCIe/NVMe/DDR 带宽远低于 HBM。on-demand loading 会让 decode 阶段从 compute-bound 变成 I/O-bound。
3. **cache locality 瓶颈**: 相邻 token/layer 有 expert reuse, 但默认 router 不一定为缓存友好; 简单 LRU 在 workload 变化时命中率有限。
4. **控制与 batching 瓶颈**: decode token 小 batch 导致 GPU kernel/transfer/control overhead 难以摊薄; CPU-only 时则受线程调度、NUMA 和 page fault 影响。

代表工作梳理

MoE-Lightning

MoE-Lightning 面向 memory-constrained GPU 上的高吞吐 MoE batch inference。核心是 **CGOPipe**: 同时调度 CPU、GPU 和 I/O, 把 weight paging、hidden states、KV cache 等事件串成流水线, 减少 bubble; 另有 **HRM** 层次化 roofline model 用于搜索高吞吐策略。论文称在单 T4 16GB 上对 Mixtral-8x7B 比已有 offloading 系统最高 10.3x throughput, 并可用 2-3x 更少 CPU memory 接近理论上限。可借鉴点是: 不要只写一个 expert loader, 要写一个端到端 pipeline scheduler 和性能模型。

本地资料: refs/papers/2411.11217.pdf , refs/pages/pschafhalter_moe-lightning.html 。

MoE-Gen

MoE-Gen 针对 offline/high-throughput, 而非低延迟 chat。它指出 model-based/continuous batching 在 MoE 的 attention/expert module 上会形成过小 batch, GPU 利用率很低。其 **module-based batching** 把 token 积累在 host memory, 再为 attention/expert 分别动态发大 batch, 并选择 batch size 以重叠计算和通信。论文报告比 FlexGen/MoE-Lightning/DeepSpeed 高 8-31x throughput。可借鉴点是: 如果我们的目标包括批量评测、数据处理、RAG embedding/rewriting, 不应沿用 chat 低延迟调度, 应支持吞吐模式。

本地资料: refs/papers/2503.09716v1.pdf 。

FloE

FloE 认为 offloading 的关键不只是“预测哪个 expert”, 还要减少每次搬运的 expert 参数量。它利用 **intra-expert redundancy** 做 contextual sparsification: 找到 up projection 中低幅、不显著的输出激活, 移除 gate/down 对应 channel, 同时对 up projection 更激进量化。再结合低成本 sparse prediction 和 prefetch。论文称 Mixtral-8x7B 每 expert 9.3x 压缩、11GB VRAM 可部署、相对 DeepSpeed-MII 最高 48.7x speedup, 平均性能下降 4.4%-7.6%。可借鉴点是: 对内存受限 CPU 场景, 专家压缩直接减少磁盘、内存和 PCIe 负担, 比只改调度更基础。

本地资料: refs/papers/2505.05950.pdf , refs/repos/zju-stu-lizheng__FloE 。

DALI

DALI 面向 local PC，认为已有 CPU-GPU 混合执行存在三类问题: 静态专家分配导致 CPU/GPU 负载不均，prefetch 不能准确预测高工作量专家，cache replacement 忽略 workload dynamics。DALI 把 CPU/GPU expert assignment 建模为 0-1 integer optimization，并用 Greedy Assignment 近似；用 **Residual-Based Prefetching** 预测高工作量专家；用 **Workload-Aware Cache Replacement** 利用 expert activation 的时间相关性。可借鉴点是: expert placement 不应只是“热门上 GPU、冷门在 CPU”，而要按当前 workload、batch、层、专家计算量和传输代价动态调整。

本地资料: refs/papers/2602.03495.pdf。

CommitMoE

CommitMoE 针对 prefetch 失败后的 fallback 成本。普通 prefetch 一旦预测错，错误 expert copy 已经占用传输通道，正确 expert 还要 on-demand load。CommitMoE 提出 **Commit Router**: 在高置信度下直接 commit 预测专家、避免 fallback；在低置信度场景，利用模型输出对 expert selection 的鲁棒性，并用 output-weight-aware 设计降低质量损失。AAAI 论文报告比已有 offloading framework 快 1.3x-9.4x 且保持质量。可借鉴点是: prefetch 不必总是“错了就完全回退”，可以按置信度把 routing 变成一个系统层决策。

本地资料: refs/papers/aaai2026_commitmoe_39454.pdf。

Speculating Experts

这篇工作用当前层内部表示预测下一层 experts。其 quasi-hidden state 来自 normalized residual stream 和 default vector，提前决定下一层专家，使 CPU-GPU transfer 与当前计算重叠。它强调 parameter-free prefetching，并进一步发现“执行 speculated experts”在不少 MoE 上可以保持下游精度，从而避免错预测后的重新加载。论文报告 TPOT 降低 5%-14%。可借鉴点是: 对 decode latency，下一层 expert prefetch 是低侵入、可先做 tracing 验证的优化。

本地资料: refs/papers/2603.19289.pdf，refs/repos/axonn-ai__yalis。

SpecMoE

arXiv 2604.10152 的 SpecMoE 是 self-assisted speculative decoding: 不另训一个大 draft model，而使用目标 MoE 自身部分作为 draft，减少开发成本和存储成本；系统层目标是减少 CPU-GPU 之间迁移的 expert 参数量，论文报告 throughput 最高提升 4.30x。JuncaiL/SpecMoE 是较早的 speculative MoE draft model 训练脚本，证明小 MoE draft 能提高 verified tokens，但 README 也指出当前实现逐 expert 顺序计算，需要 fused expert 计算才能真正低延迟。可借鉴点是: 对 MoE，speculation 要和 expert cache 联动，否则 draft/verify 可能放大 expert load。

本地资料: refs/papers/2604.10152.pdf，refs/repos/JuncaiL__SpecMoE。

CrossPool

CrossPool 针对 cold MoE multi-LLM serving。它把 FFN weights 和 KV-cache 分成两个 GPU memory pools: weights pool 合并冷模型 FFN weights, KV-cache pool 按活跃请求动态分配。再加 KV planner/virtualizer、layer-wise pipeline scheduler 和 persistent kernels/control lowering。论文报告相对 kvcached-based multi-LLM serving P99 TBT 最高降低 10.4x。可借鉴点是: 如果未来目标是同一台机器服务多个冷门 MoE 或多 tenant agent, 必须把“权重常驻需求”和“KV 瞬时需求”分离。

本地资料: refs/papers/2606.24506.pdf , refs/pages/alphaxiv_2606.24506v1.html 。

PreMoE 和 ReMoE

PreMoE 是 deployment-specific expert pruning。它用 router logits 计算 **Predicted Expert Utility (PEU)**, 通过 high-confidence filtering 和 logit transformation 得到 domain-aware expert ranking, 再编译只保留 top-M experts 的 specialist/generalist。论文称 30B-718B MoE 可到 50% sparsity 且几乎无性能损失。ReMoE 则是 router-only fine-tuning, 不改 expert weight、kernel 或 cache policy, 只调 gate/router, 让相邻 token 更倾向复用 experts, 从而提高 expert-cache locality、降低 offloading overhead。

可借鉴点是: 运行时 cache 之外, 还可以在部署前改变“会被加载的专家集合”和“router 的 locality”。对固定领域、固定 workload, 这是非常高性价比的方向。

本地资料: refs/papers/2505.17639.pdf , refs/repos/JarvisPei__PreMoe ,
refs/repos/BUAA-OSCAR__ReMoE 。

MoE-Infinity, mixtral-offloading, CPU-GPU collaborative inference

MoE-Infinity 是比较完整的生产化 offload runtime: expert weights 放 CPU/SSD, activation-aware cache 保持 hot experts, hooks/tracing/prefetch 隐藏传输, 服务层支持 continuous batching、paged KV cache、streaming、preemption、prefix cache、KV offload coordinator、LRU/ARC policy 和 multi-GPU expert distribution。mixtral-offloading 是更早更小的实现, 组合 HQQ mixed quantization 和 per-layer expert LRU。elsa-lab 的 CPU-GPU collaborative inference 使用 N-index, M-way set-associative GPU expert cache, cache miss 时 CPU 先算, 同时后台 fetch, 利用 consecutive layer reuse 约 44% 和 consecutive token reuse 40%-60%, README 声称相对 prefetching 方法最高 4.4x speedup。

可借鉴点是: 工程上先做简单 LRU 能跑, 再逐步升级为 set-associative/ARC/workload-aware; miss 处理不要只阻塞等待 GPU fetch, CPU compute fallback 可以把 miss 从硬阻塞变成可重叠路径。

本地资料: refs/repos/EfficientMoE__MoE-Infinity ,
refs/repos/dvmazur__mixtral-offloading ,
refs/repos/elsa-lab__MoE-CPU-GPU-Collaborative-Inference 。

colibri, KTransformers, vllm-Moet, InstinctRazor

colibri 是 CPU-first/CPU-only 风格最接近的工程样本: dense path 常驻 RAM int4, 19,456 routed experts 在磁盘上 int4 存放, 按需 streaming, per-layer LRU、pinned hot-store、OS page cache 作为 L2; 还实现 async expert readahead、router-lookahead prefetch、batch-union MoE、MTP speculative decoding、grammar-forced draft、RAM safety sizing、AVX2 integer-dot kernels。README 给出的关键教训是: int8 MTP head 才有 39%-59% acceptance, int4 draft head acceptance 可能降到 0%-4%; 冷 cache 下 speculation 会增加 expert-loads/token, 可能变慢。

KTransformers 提供 CPU-GPU heterogeneous inference、CPU AMX/AVX INT4/INT8 kernels、NUMA-aware MoE、GPU expert mask/scheduling, 并能和 SGLang 集成。它适合作为 CPU kernel 与异构 placement 的参考。vllm-Moet 展示了更激进路线: 2-bit routed experts + FP4 recovery/delta cache/confidence gate、GPU/pinned RAM/NVMe 三层 expert residency、NVFP4 KV cache、MTP、CUDA graphs。InstinctRazor 则强调 sub-4-bit quantization: INT3 experts、INT4 backbone、BF16 保护 router/norm/vision, OPD/on-policy distillation 恢复精度。

可借鉴点是: CPU 场景首先需要专门的低比特 matmul 和 expert storage format; 如果上 GPU, tiered residency 和精度恢复池比“全部专家同精度”更现实。

本地资料: refs/repos/JustVugg__colibri, refs/repos/kvcache-ai__KTransformers, refs/repos/kacper-daftcode__vllm-Moet, refs/repos/General-Instinct__InstinctRazor。

技术手段归纳

1. 分层常驻策略

推荐把模型拆成:

- **Always-on resident path:** embedding、attention、norm、router、shared expert、lm_head。优先常驻 CPU RAM 或 GPU VRAM, 低比特时也要保护关键模块。
- **Routed expert store:** 每层每专家独立切片, 支持 mmap、sequential layout、packed low-bit、per-row/per-group scale、校验和、版本化 metadata。
- **Hot expert cache:** CPU-only 可用 RAM cache + OS page cache; CPU+GPU 加 GPU resident expert cache, 命中直接跑 GPU kernel, miss 进入 CPU compute 或 async transfer。
- **KV store:** paged/compressed KV 单独管理, 不要和 expert cache 争同一预算; 长会话可持久化 KV。

这对应 colibri 的 dense resident + disk experts, MoE-Infinity 的 expert cache + KV coordinator, CrossPool 的 weight/KV disaggregation, vllm-Moet 的 GPU/pinned RAM/NVMe 三层。

2. 专家低比特与结构压缩

可组合的做法:

- INT4 作为第一版专家格式, 成熟、CPU/GPU 都有参考实现。
- INT3/2-bit routed experts 作为第二阶段, 必须保护 router/norm/shared path, 并准备 FP4/INT4 recovery 或 LoRA/OPD recovery。
- 按专家/按 layer/按 projection 分配 bit-width。不要只用 activation frequency, PreMoE/InstinctRazor 的经验显示 router logit utility、salience、protected path 更稳。
- FloE 式 intra-expert channel sparsification 适合降低搬运量, 但需要校准和质量回归。
- PEU pruning 适合固定领域部署, 把全量模型预编译成 specialist/generalist。

一个务实 recipe: router/norm/shared expert 至少 FP16/BF16 或 INT8; attention/backbone INT4; routed experts INT4 起步, 逐步试 INT3; 低于 3 bit 时做精度恢复, 不要只看困惑度, 要看任务质量和 routing flip。

3. Cache 与 replacement policy

从低到高:

1. per-layer LRU: 实现简单, 是 baseline。
2. set-associative cache: 限制单层/单组被挤出, 提高稳定性。
3. ARC/LRU-K/workload-aware: 同时考虑最近访问、频率、token/layer temporal correlation、专家计算量和传输成本。
4. PEU/static pruning + runtime cache: 先减少候选专家, 再缓存高频专家。
5. Router locality training: ReMoE 这类 router-only fine-tuning 可从源头提高 reuse。

Cache 的评测必须区分 cold/warm。MoE-Infinity 文档也提示: 第一次请求包含 disk -> CPU -> GPU 加载, 后续才体现 expert cache。

4. Prefetch 与 commit execution

可用信号:

- 当前层 post-attention hidden state 或 quasi-hidden state 预测下一层 routing。
- residual-based predictor 预测高 workload experts。
- router logits 的 certainty 决定是否提前 commit。
- batch/prompt/domain PEU 排名决定开局预热。

推荐策略:

- 第一版只做 speculative prefetch, 不改变实际 routing。
- 第二版在高置信度时允许 commit execution, 低置信度 fallback。
- 对错预测记录三类指标: wasted bytes、saved stall time、quality delta。

- 对 CPU-only，把 prefetch 解释为 disk/OS page cache readahead；对 CPU+GPU，把 prefetch 解释为 pinned host -> GPU async copy。

5. Pipeline、batching 与调度

推理系统要同时支持两种模式：

- **interactive mode**: 目标是低 ITL/TPOT，batch 小，需要 next-layer prefetch、hot cache、MTP、KV 压缩。
- **throughput mode**: 目标是 token/s，允许 host memory 累积 token，适合 MoE-Gen 的 module-based batching、batch-union MoE、continuous batching。

调度层需要：

- double-buffer: 当前 expert 计算时加载下一批 expert。
- 多线程/多 stream: disk I/O、CPU dequant、GPU H2D、GPU compute 分离。
- batch-union MoE: prefill 或 MTP verification 时，同一 batch 内相同 expert 只加载一次。
- layer-wise pipeline: attention/FFN/KV/hidden-state transfer 拆开排。
- admission control: 按 expert cache/KV budget 而不只是请求数限流。

6. Speculative decoding

可用来源：

- 模型原生 MTP head。
- self-assisted draft: 用目标 MoE 的部分层/子模型 draft。
- 小 dense/MoE draft model。
- grammar-forced draft: JSON/工具调用/结构化抽取时特别有效。

注意事项：

- Speculation 会改变每 step 的 batch 形状，低比特 kernel 可能产生非 byte-identical 输出；需要定义“分布正确”还是“贪心字节级一致”。
- 冷 cache 时 draft/verify 可能导致更多专家加载，先做 warm-cache 条件下的 A/B。
- MTP head 不应过度量化；colibri 的经验是 int4 MTP head acceptance 可能崩掉。
- 自助 speculative decoding 需要和 expert cache 合并统计，否则论文 speedup 不一定在 CPU-only/磁盘场景复现。

7. KV-cache 与多模型服务

长上下文下，KV-cache 会和 expert cache 抢内存。建议：

- paged KV 是 baseline。
- MLA/MQA/GQA 或模型原生 compressed KV 要优先利用。

- KV 和 expert weight 分池管理，不共用一个 OOM 阈值。
- 多冷模型服务时采用 CrossPool 思路: 静态 weights pool + 动态 KV pool。
- 对本地 agent 场景，KV persistence 可以避免多轮会话重复 prefill，但要处理版本/采样/系统 prompt 变更。

如果我们做这个方向: 推荐路线

Phase 0: 明确目标和基准

先选一个模型族和硬件档:

- CPU-only: 16-64 cores, 32-256GB RAM, NVMe。
- CPU+小 GPU: 24GB/32GB/48GB VRAM + 128-512GB RAM。
- 模型候选: Mixtral-8x7B、DeepSeek-V2-Lite、Qwen3-30B-A3B、GLM/DeepSeek 新 MoE 的小配置。

必须建立这些指标:

- TTFT、TPOT/ITL、decode token/s、prefill token/s。
- resident RAM、peak RAM、VRAM、disk read MB/token、PCIe H2D MB/token。
- expert cache hit rate、miss stall、wasted prefetch bytes。
- 质量: 至少做困惑度/短 eval/任务集，并记录 routing flip。
- cold cache、warm cache、长会话、批量任务分别测。

Phase 1: CPU-first baseline runtime

目标是“稳定跑起来”:

- 离线 converter: HF/safetensors -> per-layer/per-expert packed store。
- INT4 expert format + scales + metadata; 先不要追求 INT3/2-bit。
- Always-on path 常驻 RAM; routed experts mmap/stream。
- per-layer LRU expert cache, 自动根据 available memory sizing。
- AVX2/AVX512/AMX 后端优先复用 KTransformers/llama.cpp/ggml 风格 kernel。
- prefill 做 batch-union: 同一 batch 内一个 expert 只读一次。

Phase 2: 异步 I/O 和 cache scheduler

目标是让 decode 不被 disk/PCIe 完全阻塞:

- I/O thread pool + request queue + priority。
- sequential expert pack + readahead, 避免随机小文件 I/O。
- double-buffer: 当前层 compute 时读取下一层候选 experts。
- CPU+GPU 时引入 pinned host arena、GPU expert cache、async H2D stream。

- miss policy: GPU miss 时可以 CPU compute fallback, 同时后台 fetch 到 GPU。

Phase 3: 预测和自适应 placement

目标是减少 miss:

- 记录真实 routing traces, 先离线评估 next-layer prediction upper bound。
- 实现 residual/quasi-hidden predictor 或 router-lookahead。
- 引入置信度阈值: 低置信度只 prefetch, 高置信度可 commit。
- 实现 workload-aware CPU/GPU expert assignment, 至少用简单 greedy: 比较 CPU compute time 与 H2D+GPU compute time。
- Cache replacement 从 LRU 升级到 LRU-K/ARC/workload-aware。

Phase 4: 压缩、剪枝与 locality 优化

目标是把内存和 I/O 压到可接受:

- INT3 routed experts + protected always-on path。
- 对高价值 experts 保留 INT4/FP4 recovery, 对低价值 experts 用 INT3/2-bit。
- PEU/PreMoE: 固定领域先做 expert ranking, 试 25%-50% expert pruning。
- ReMoE: 若可训练, 尝试 router-only fine-tuning 提高短窗口 expert reuse。
- FloE 风格 channel sparsification 可作为论文创新点, 但工程复杂度更高。

Phase 5: Speculation 和 serving

目标是产品化:

- 原生 MTP head 优先, 保持 head 精度。
- 无 MTP 时用 self-assisted draft 或小 draft model。
- 结构化输出接 grammar-forced draft。
- 服务层支持 continuous batching、paged KV、prefix/KV persistence、请求抢占。
- 多模型冷启动/冷模型服务时, 引入 weight/KV disaggregation。

最可能形成差异化的方向

1. **CPU-first expert cache runtime**: 很多工作默认有小 GPU; 纯 CPU/NVMe/RAM 场景仍有空间。colibri 是强参考, 但可以做更通用的模型抽象、量化格式和 tracing。
2. **Cache-aware routing adaptation**: ReMoE + runtime cache 的闭环值得做。目标不是提升模型能力, 而是把 expert reuse 变成可优化指标。
3. **PEU pruning + runtime cache 联动**: PreMoE 只做静态 compile, runtime cache 只看近期访问。把 domain PEU、实时 hotness、cache budget 合成一个 residency policy 可能有工程价值。

- 4. **Fallback-free/commit prefetch 在 CPU-only 上的版本**: CommitMoE 面向 CPU-GPU offload；在磁盘/内存流式 CPU 推理里，错误 readahead 的代价也很高，置信度 commit 可以转化为“先算预测专家，必要时再修正”的策略。
- 5. **结构化任务专用 speculation**: grammar-forced draft 对 JSON/tool calling 很实用，且比训练 draft model 便宜，适合 agent 本地推理。

风险和坑

- 只看 **warm cache 会高估速度**。需要 cold/warm/mixed 三套指标。
- **低比特专家会影响 routing 间接质量**。即使 router 保持 BF16，expert 输出变化也会改变后续 hidden state 和 routing。
- **MTP/speculation 不保证字节级一致**。低比特 kernel 的 shape-dependent rounding 会改变贪心输出。
- **CPU NUMA 和 page cache 很关键**。大模型 CPU-only 不是 Python 层能调好的，需要 pinning、huge page、sequential pack、prefetch hints。
- **小 batch decode 难以吃满 GPU**。没有 batching/pipeline，GPU hot cache 也可能不够快。
- **TeraXj78/SpecMoE 同名但不相关**。不要把 EEG MoE foundation model 当成 LLM inference optimization。
- **GitHub README 中的新模型性能声明需要复验**。vllm-Moet、colibri、InstinctRazor 等工程很有价值，但指标应按我们的硬件和 workload 重测。

建议的最小可行实验

实验	目标	成功标准
INT4 expert store + per-layer LRU	跑通 CPU-only MoE decode	质量基本保持，稳定生成，无 OOM
Cache warm/cold A/B	明确瓶颈来自 disk、RAM、PCIe 还是 compute	输出 MB/token、hit rate、TPOT 分解
Batch-union prefill	降低 prefill 重复 expert load	相同 prompt batch 下 expert reads/token 明显下降
Router-lookahead prefetch	验证下一层 expert 可预测性	prefetch saved stall > wasted bytes
GPU hot cache + CPU fallback	小 GPU 加速 decode	miss 不再硬阻塞，warm TPOT 明显下降
PEU pruning 25%/50%	减少常驻/可选 expert 集	任务质量下降可控，内存/读盘同比下降

实验	目标	成功标准
MTP/grammar speculation	加速结构化或长回答生成	warm cache 下 tokens/forward 提升，质量回归通过

最终判断

这个方向值得做，但不要从“再写一个 vLLM patch”开始。更稳的路线是先做 **可观测的 memory hierarchy runtime**: 低比特 expert store、cache、异步流水线、prefetch trace、质量回归。等瓶颈被量化后，再把 PreMoE/ReMoE/FloE/CommitMoE/SpecMoE 这类更激进技术逐个接入。

若目标是内存受限 CPU 场景，最优先的技术手段是:

- 1. expert 低比特格式和 always-on path 保护；
- 2. per-layer LRU/ARC + OS page cache/pinned hot-store；
- 3. async readahead 和 batch-union MoE；
- 4. router/residual lookahead prefetch；
- 5. PEU/static pruning 和 router locality；
- 6. MTP/grammar speculation；
- 7. 小 GPU 可用时再加 GPU expert cache、CPU fallback 和 KV/weight disaggregation。